IBERO CIUDAD DE MÉXICO

Ingeniería Electrónica /

Universidad Iberoamericana

Lógica Difusa

Calculadora de propina

Sistema de inferencia difusa Mamdani: tiempo de servicio y precio

Práctica: Sistema de Inferencia Difusa

Alumno: *Carlos Ortega*
Profesores: Dr. Alexandro López González
Mtro. Julio Caballero Mora
Fecha: 9 de noviembre de 2026

Calculadora: `https://sebastiux.github.io/fuzzylogicrestaurante/`
Código: `https://github.com/sebastiux/fuzzylogicrestaurante`

Índice

1. Introducción **2**

2. Objetivos **2**

3. Marco teórico **2**
3.1. Conjuntos difusos . . . 2
3.2. Funciones de pertenencia . . . 3
3.3. Operadores difusos . . . 3
3.4. Inferencia Mamdani . . . 3

4. Planteamiento del problema **3**
4.1. Relación entre precio y tiempo . . . 3
4.2. Definición del sistema . . . 4
4.3. Base de reglas . . . 4

5. Configuración en MATLAB **5**
5.1. Fuzzy Logic Designer paso a paso . . . 5
5.2. Definición por código . . . 5
5.3. Archivo `propina.fis` . . . 6
5.4. Funciones de pertenencia . . . 6

6. Calculadora web **7**
6.1. Arquitectura . . . 7
6.2. Interfaz . . . 9

7. Resultados paso a paso **11**
7.1. Caso 1: tiempo = 15 min, precio = \$800 . . . 11
7.2. Caso 2: tiempo = 42 min, precio = \$1,500 . . . 13

8. Superficie de control y casos de prueba **14**
8.1. Verificación . . . 15

9. Obtención de datos en MATLAB **15**

10. Conclusiones **16**

1 Introducción

Decidir cuánta propina dejar es un problema de razonamiento aproximado: nadie califica el servicio con un número exacto, sino con expresiones como *"tardaron bastante"* o *"la cuenta salió cara"*. La lógica difusa permite trabajar con este tipo de conceptos vagos, asignando a cada valor numérico un *grado de pertenencia* entre 0 y 1 a conjuntos lingüísticos, y combinarlos con reglas del tipo *si–entonces* para obtener una salida numérica.

En esta práctica se diseña un **sistema de inferencia difusa tipo Mamdani** con dos entradas, el **tiempo de servicio** y el **precio de la cuenta**, y una salida, el **porcentaje de propina recomendado**. El sistema se define con el *Fuzzy Logic Designer* de MATLAB y se exporta al archivo `propina.fis`. Ese mismo archivo es leído por una calculadora web publicada en GitHub Pages, que replica la inferencia de `evalfis` y muestra paso a paso cómo se obtiene la recomendación.

Calculadora en línea: `https://sebastiux.github.io/fuzzylogicrestaurante/` **Código fuente:** `https://github.com/sebastiux/fuzzylogicrestaurante`

2 Objetivos

General. Diseñar e implementar un sistema de inferencia difusa Mamdani que recomiende el porcentaje de propina a partir de la relación entre el tiempo de servicio y el precio de la cuenta.

Específicos.

1. Definir las variables lingüísticas, sus universos de discurso y sus funciones de pertenencia en el Fuzzy Logic Designer de MATLAB.
2. Construir una base de reglas que refleje la relación precio–tiempo: una cuenta cara justifica una espera mayor que una barata.
3. Obtener los datos del sistema con `evalfis`: grados de pertenencia, fuerza de disparo de las reglas, conjunto agregado y salida defuzzificada.
4. Publicar una calculadora web que use el mismo archivo `.fis` y verificar que sus resultados coinciden con una ejecución independiente del sistema.

3 Marco teórico

3.1 Conjuntos difusos

Un conjunto difuso A sobre un universo X se define por su función de pertenencia $\mu_A : X \to [0, 1]$, que indica en qué grado cada x pertenece a A [1]. A diferencia de un conjunto clásico, un mismo valor puede pertenecer parcialmente a varios conjuntos; por ejemplo, 15 minutos de servicio son a la vez *rápido* en grado 0.5 y *normal* en grado 0.33.

3.2 Funciones de pertenencia

En este sistema se usan las funciones triangular y trapezoidal, con la misma definición que el Fuzzy Logic Toolbox [4]:

$$\text{trimf}(x; a, b, c) = \text{máx}\left(\text{mín}\left(\frac{x-a}{b-a},\ \frac{c-x}{c-b}\right),\ 0\right), \tag{1}$$

$$\text{trapmf}(x; a, b, c, d) = \text{máx}\left(\text{mín}\left(\frac{x-a}{b-a},\ 1,\ \frac{d-x}{d-c}\right),\ 0\right). \tag{2}$$

Cuando $a = b$ (o $c = d$) el trapecio tiene un hombro: vale 1 desde el inicio (o hasta el final) del universo.

3.3 Operadores difusos

La conjunción (AND) se modela con el mínimo y la disyunción (OR) con el máximo:

$$\mu_{A\wedge B}(x) = \text{mín}\left(\mu_A(x),\ \mu_B(x)\right), \qquad \mu_{A\vee B}(x) = \text{máx}\left(\mu_A(x),\ \mu_B(x)\right). \tag{3}$$

3.4 Inferencia Mamdani

En un sistema Mamdani [2] cada regla tiene la forma *si* x_1 *es* A_i *y* x_2 *es* B_i *entonces* y *es* C_i, con un conjunto difuso en el consecuente. La inferencia tiene cuatro etapas [3]:

1. **Fuzzificación.** Se evalúa el grado de pertenencia de cada entrada a sus conjuntos: $\mu_{A_i}(x_1)$, $\mu_{B_i}(x_2)$.

2. **Fuerza de disparo.** Se combinan los antecedentes con el operador AND: $w_i = \text{mín}\left(\mu_{A_i}(x_1), \mu_{B_i}(x_2)\right)$.

3. **Implicación y agregación.** Cada consecuente se recorta a la altura w_i y los conjuntos recortados se unen con el máximo:
$$\mu_{\text{agr}}(y) = \max_i\ \text{mín}\left(w_i,\ \mu_{C_i}(y)\right). \tag{4}$$

4. **Defuzzificación.** El conjunto agregado se convierte en un número con el centroide. MATLAB discretiza el universo de salida en $N = 101$ puntos y_k y calcula
$$y^* = \frac{\sum_{k=1}^{N} y_k\, \mu_{\text{agr}}(y_k)}{\sum_{k=1}^{N} \mu_{\text{agr}}(y_k)}. \tag{5}$$

4 Planteamiento del problema

4.1 Relación entre precio y tiempo

La idea que guía el diseño es que la tolerancia a la espera depende de lo que se está pagando. En una cuenta cara es razonable que los platillos tarden más, porque suelen ser más elaborados; en una cuenta barata, esperar mucho es una mala experiencia. Por eso un mismo tiempo *lento* se castiga menos cuando el precio es alto, y un servicio *rápido* en una cuenta cara merece la propina más alta.

4.2 Definición del sistema

Tabla 1: Parámetros generales del sistema de inferencia difusa.

Propiedad	Valor
Nombre	`propina`
Tipo	Mamdani
Entradas	`tiempo` (min) en $[0, 60]$ y `precio` (MXN) en $[0, 2000]$
Salida	`propina` (%) en $[0, 25]$
AND / OR	`min` / `max`
Implicación / Agregación	`min` / `max`
Defuzzificación	`centroid` (101 puntos)
Reglas	9, todas con conector AND y peso 1

Tabla 2: Funciones de pertenencia de las tres variables.

Variable	Rango	Conjunto	Tipo	Parámetros
`tiempo` (min)	$[0, 60]$	*rapido*	trapmf	[0 0 10 20]
		normal	trimf	[10 25 40]
		lento	trapmf	[30 45 60 60]
`precio` (MXN)	$[0, 2000]$	*barato*	trapmf	[0 0 200 600]
		moderado	trimf	[300 800 1300]
		caro	trapmf	[1000 1500 2000 2000]
`propina` (%)	$[0, 25]$	*muy_baja*	trapmf	[0 0 3 6]
		baja	trimf	[3 7 11]
		media	trimf	[9 12.5 16]
		alta	trimf	[14 18 22]
		muy_alta	trapmf	[20 23 25 25]

Los conjuntos de cada entrada se traslapan, de modo que cualquier valor del universo activa al menos un conjunto y ninguna combinación de entradas deja al sistema sin reglas activas.

4.3 Base de reglas

Tabla 3: Base de reglas en forma de matriz: cada celda es el consecuente de la regla *si tiempo es (fila) y precio es (columna)*.

tiempo \ precio	*barato*	*moderado*	*caro*
rapido	*alta* (R1)	*alta* (R2)	*muy_alta* (R3)
normal	*media* (R4)	*media* (R5)	*alta* (R6)
lento	*muy_baja* (R7)	*baja* (R8)	*media* (R9)

En cada columna la propina baja conforme el servicio es más lento, y en cada fila sube (o se mantiene) conforme el precio es mayor; ésta es la relación precio–tiempo descrita arriba.

5 Configuración en MATLAB

5.1 Fuzzy Logic Designer paso a paso

1. Abrir la aplicación con `fuzzyLogicDesigner` (en R2022b o anteriores, `fuzzy`) y crear un sistema **Mamdani** con 2 entradas y 1 salida.
2. Renombrar las variables a `tiempo`, `precio` y `propina`, y capturar el *Range* de cada una (tabla 2).
3. Para cada variable, editar o agregar sus funciones de pertenencia con el nombre, tipo y parámetros de la tabla 2. La salida necesita cinco funciones.
4. Fijar los métodos del sistema: And `min`, Or `max`, Implication `min`, Aggregation `max`, Defuzzification `centroid`.
5. Capturar las nueve reglas de la tabla 3 con conector *and* y peso 1.
6. Verificar en el visor de reglas con la entrada [15 800]: la propina debe ser 15.89 %.
7. Exportar con *Export* → *To File* como `propina.fis`.

El repositorio incluye `matlab/abrir_designer.m`, que abre el diseñador con todo configurado, y la guía `matlab/GUIA_DESIGNER.md` con cada campo a llenar.

5.2 Definición por código

El mismo sistema se puede construir con la API del Fuzzy Logic Toolbox (R2018b o posterior). El código 1 resume `matlab/crear_fis_propina.m`.

Código 1: Construcción del FIS Mamdani por código.

```
fis = mamfis('Name','propina','AndMethod','min','OrMethod','max', ...
    'ImplicationMethod','min','AggregationMethod','max', ...
    'DefuzzificationMethod','centroid');

fis = addInput(fis,[0 60],'Name','tiempo');
fis = addMF(fis,'tiempo','trapmf',[0 0 10 20],'Name','rapido');
fis = addMF(fis,'tiempo','trimf',[10 25 40],'Name','normal');
fis = addMF(fis,'tiempo','trapmf',[30 45 60 60],'Name','lento');

fis = addInput(fis,[0 2000],'Name','precio');
fis = addMF(fis,'precio','trapmf',[0 0 200 600],'Name','barato');
fis = addMF(fis,'precio','trimf',[300 800 1300],'Name','moderado');
fis = addMF(fis,'precio','trapmf',[1000 1500 2000 2000],'Name','caro');

fis = addOutput(fis,[0 25],'Name','propina');
fis = addMF(fis,'propina','trapmf',[0 0 3 6],'Name','muy_baja');
fis = addMF(fis,'propina','trimf',[3 7 11],'Name','baja');
fis = addMF(fis,'propina','trimf',[9 12.5 16],'Name','media');
fis = addMF(fis,'propina','trimf',[14 18 22],'Name','alta');
fis = addMF(fis,'propina','trapmf',[20 23 25 25],'Name','muy_alta');

% [tiempo precio propina peso conector(1 = AND)]
reglas = [1 1 4 1 1; 1 2 4 1 1; 1 3 5 1 1;
          2 1 3 1 1; 2 2 3 1 1; 2 3 4 1 1;
          3 1 1 1 1; 3 2 2 1 1; 3 3 3 1 1];
fis = addRule(fis, reglas);
writeFIS(fis,'propina');
```

5.3 Archivo `propina.fis`

El código 2 muestra las secciones principales del archivo exportado; es el que lee la calculadora web.

Código 2: Extracto de `matlab/propina.fis`.

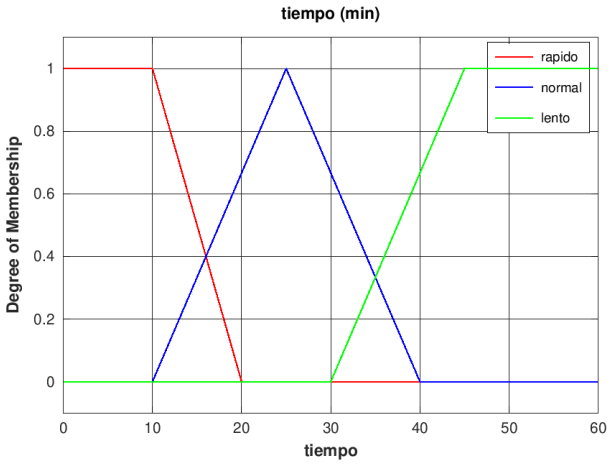
```
[System]
Name='propina'
Type='mamdani'
NumInputs=2
NumOutputs=1
NumRules=9
AndMethod='min'
OrMethod='max'
ImpMethod='min'
AggMethod='max'
DefuzzMethod='centroid'

[Input1]
Name='tiempo'
Range=[0 60]
NumMFs=3
MF1='rapido':'trapmf',[0 0 10 20]
MF2='normal':'trimf',[10 25 40]
MF3='lento':'trapmf',[30 45 60 60]

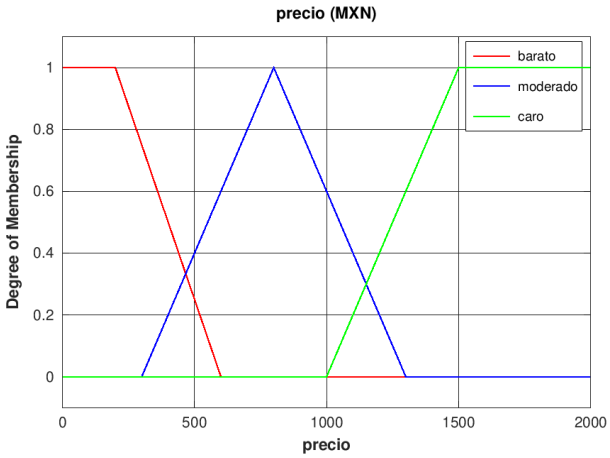
[Rules]
1 1, 4 (1) : 1
1 2, 4 (1) : 1
1 3, 5 (1) : 1
...
3 3, 3 (1) : 1
```

5.4 Funciones de pertenencia

Las figuras 1 y 2 se obtuvieron ejecutando `plotmf` sobre `propina.fis` en GNU Octave 8.4 con el paquete `fuzzy-logic-toolkit` [5], compatible con la sintaxis de MATLAB (script `reporte/octave/verificar_octave.m`).



(a) Tiempo de servicio.



(b) Precio de la cuenta.

Figura 1: Funciones de pertenencia de las entradas (`plotmf`).

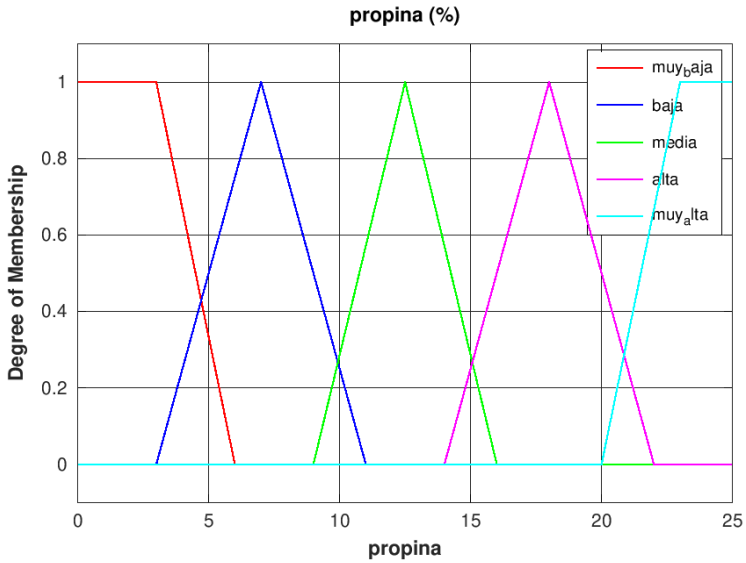


Figura 2: Funciones de pertenencia de la salida `propina` (`plotmf`).

6 Calculadora web

6.1 Arquitectura

La calculadora es una página estática (HTML, CSS y JavaScript con módulos ES, sin dependencias externas) publicada en GitHub Pages:

- **Motor difuso** (`js/fuzzy.js`): lee el archivo `.fis` de MATLAB y replica `evalfis` para sistemas Mamdani: las mismas definiciones de `trimf`, `trapmf` y otras funciones, los operadores `min`/`max`/`prod`/`probor`, la agregación y la defuzzificación sobre 101 puntos.
- **Interfaz** (`js/app.js`): controles de entrada, resultado, gráficas de fuzzificación y del conjunto agregado, tabla de reglas con su fuerza de disparo y superficie de control.

El diseño, la programación y la verificación del sistema se hicieron con apoyo del asistente de inteligencia artificial Claude [6]. Como la página carga `matlab/propina.fis` al iniciar, cualquier cambio exportado desde el Fuzzy Logic Designer y subido al repositorio se refleja en la calculadora sin modificar el código.

Algoritmo 1 Inferencia Mamdani implementada en la calculadora

Entrada: FIS leído de `propina.fis`, entradas (x_1, x_2)
Salida: Porcentaje de propina y^*

Saturar cada x_j a su rango
for cada regla i **do**
    $w_i \leftarrow \min\left(\mu_{A_i}(x_1),\ \mu_{B_i}(x_2)\right)$ ▷ fuerza de disparo
end for
$y_k \leftarrow 101$ puntos equiespaciados en $[0, 25]$
for cada y_k **do**
    $\mu_{\text{agr}}(y_k) \leftarrow \max_i \min\left(w_i,\ \mu_{C_i}(y_k)\right)$ ▷ implicación y agregación
end for
$y^* \leftarrow \sum_k y_k\, \mu_{\text{agr}}(y_k) \big/ \sum_k \mu_{\text{agr}}(y_k)$ ▷ centroide
return y^*

6.2 Interfaz

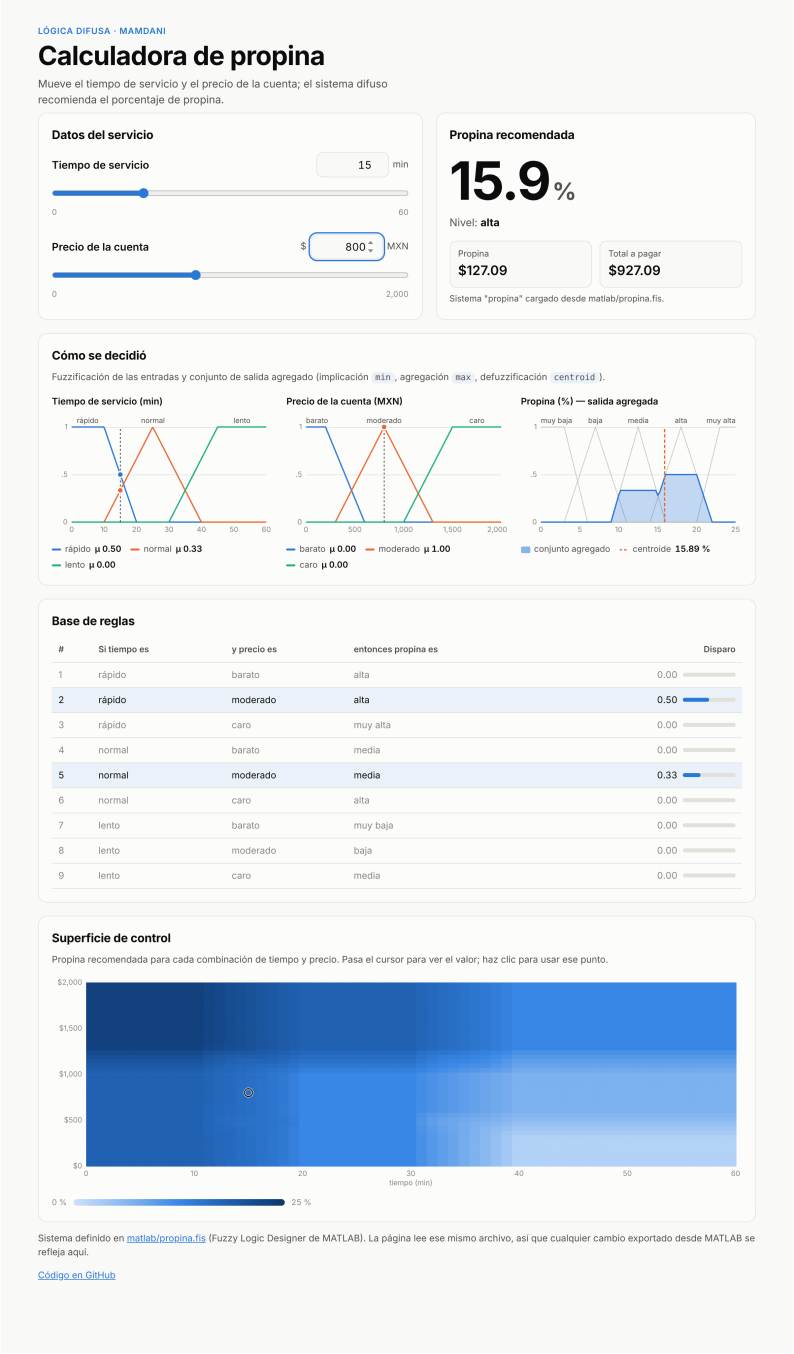


Figura 3: Vista completa de la calculadora con tiempo = 15 min y precio = $800.

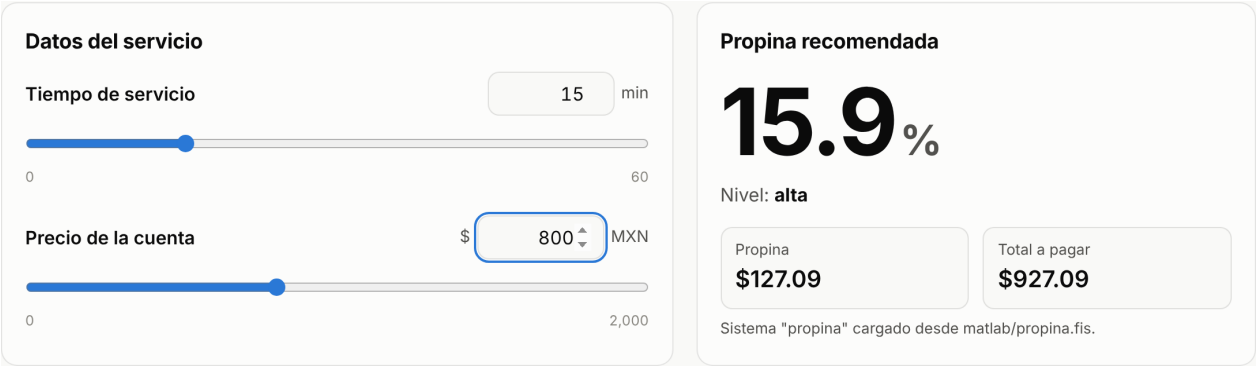


Figura 4: Entradas y resultado: propina recomendada, nivel lingüístico, monto de la propina y total a pagar.

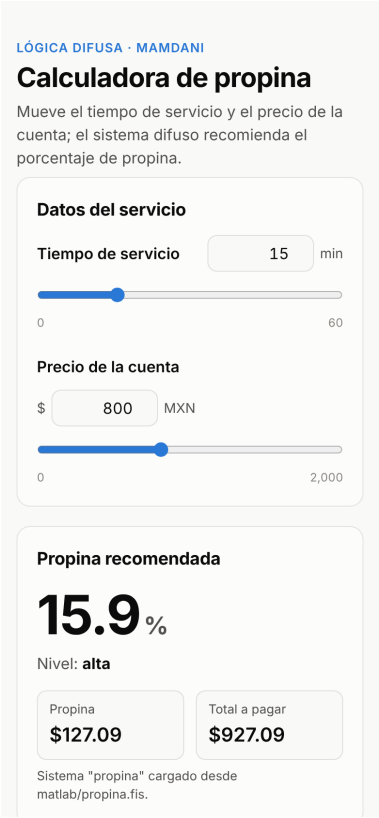


Figura 5: La calculadora en un teléfono.

7 Resultados paso a paso

7.1 Caso 1: tiempo = 15 min, precio = $800

Paso 1: fuzzificación. Con las funciones de la tabla 2:

$$\mu_{\text{rapido}}(15) = \frac{20-15}{20-10} = 0.50, \qquad \mu_{\text{normal}}(15) = \frac{15-10}{25-10} = 0.33, \qquad \mu_{\text{lento}}(15) = 0,$$
$$\mu_{\text{barato}}(800) = 0, \qquad \mu_{\text{moderado}}(800) = 1, \qquad \mu_{\text{caro}}(800) = 0.$$

Quince minutos son "algo rápido y algo normal", y $800 es plenamente "moderado".

Paso 2: fuerza de disparo. Cada regla toma el mínimo de sus antecedentes. Sólo se activan las reglas que combinan *rapido* o *normal* con *moderado*:

Tabla 4: Fuerza de disparo de las nueve reglas para la entrada [15 800].

Regla	tiempo	precio	propina	μ(tiempo)	μ(precio)	w_i = mín
R1	*rapido*	*barato*	*alta*	0.50	0	0
R2	*rapido*	*moderado*	*alta*	0.50	1	**0.50**
R3	*rapido*	*caro*	*muy_alta*	0.50	0	0
R4	*normal*	*barato*	*media*	0.33	0	0
R5	*normal*	*moderado*	*media*	0.33	1	**0.33**
R6	*normal*	*caro*	*alta*	0.33	0	0
R7	*lento*	*barato*	*muy_baja*	0	0	0
R8	*lento*	*moderado*	*baja*	0	1	0
R9	*lento*	*caro*	*media*	0	0	0

Paso 3: implicación y agregación. El conjunto *alta* se recorta a 0.50 y *media* a 0.33; su unión con el máximo da el conjunto agregado:

$$\mu_{\text{agr}}(y) = \text{máx}\Big(\text{mín}\left(0.50,\ \mu_{\text{alta}}(y)\right),\ \text{mín}\left(0.33,\ \mu_{\text{media}}(y)\right)\Big).$$

Tabla 5: Valores del conjunto agregado en algunos puntos del universo de salida.

y (%)	9	10	11	12.5	14	15	15.5	16	20	22
$\mu_{\text{agr}}(y)$	0	0.286	0.333	0.333	0.333	0.286	0.375	0.500	0.500	0

Paso 4: defuzzificación. Con los 101 puntos $y_k = 0, 0.25, \ldots, 25$ y la ecuación (5):

$$y^* = \frac{\sum_k y_k\, \mu_{\text{agr}}(y_k)}{\sum_k \mu_{\text{agr}}(y_k)} = \frac{297.1979}{18.7083} = \mathbf{15.89}\ \%.$$

El resultado queda entre *media* y *alta*, más cerca de *alta* porque esa regla dispara con más fuerza. Para una cuenta de $800 la propina es de $127.09 y el total, $927.09 (figura 4).



Figura 6: Fuzzificación y conjunto agregado en la calculadora para [15 800]. La línea punteada naranja es el centroide.

Base de reglas

#	Si tiempo es	y precio es	entonces propina es	Disparo
1	rápido	barato	alta	0.00
2	rápido	moderado	alta	0.50
3	rápido	caro	muy alta	0.00
4	normal	barato	media	0.00
5	normal	moderado	media	0.33
6	normal	caro	alta	0.00
7	lento	barato	muy baja	0.00
8	lento	moderado	baja	0.00
9	lento	caro	media	0.00

Figura 7: Base de reglas en la calculadora: se resaltan R2 ($w = 0.50$) y R5 ($w = 0.33$).



Figura 8: Visor de reglas para [15 800] generado en Octave con las salidas de `evalfis`: en cada fila, los antecedentes (área sombreada hasta el grado de pertenencia) y el consecuente recortado; abajo, el conjunto agregado y el centroide.

7.2 Caso 2: tiempo = 42 min, precio = $1,500

Ahora el servicio es lento, $\mu_{\text{lento}}(42) = \frac{42-30}{45-30} = 0.80$, y la cuenta es cara, $\mu_{\text{caro}}(1500) = 1$. Sólo dispara R9 (*lento y caro* $\Rightarrow$ *media*) con $w_9 = 0.80$. El conjunto agregado es *media* recortado a 0.80, simétrico alrededor de 12.5, así que

$$y^* = \frac{167.8571}{13.4286} = \mathbf{12.50}\ \%.$$

Este caso muestra la relación precio–tiempo: con la misma lentitud y una cuenta barata la regla activa sería R7 y la propina caería a 2.27 %.

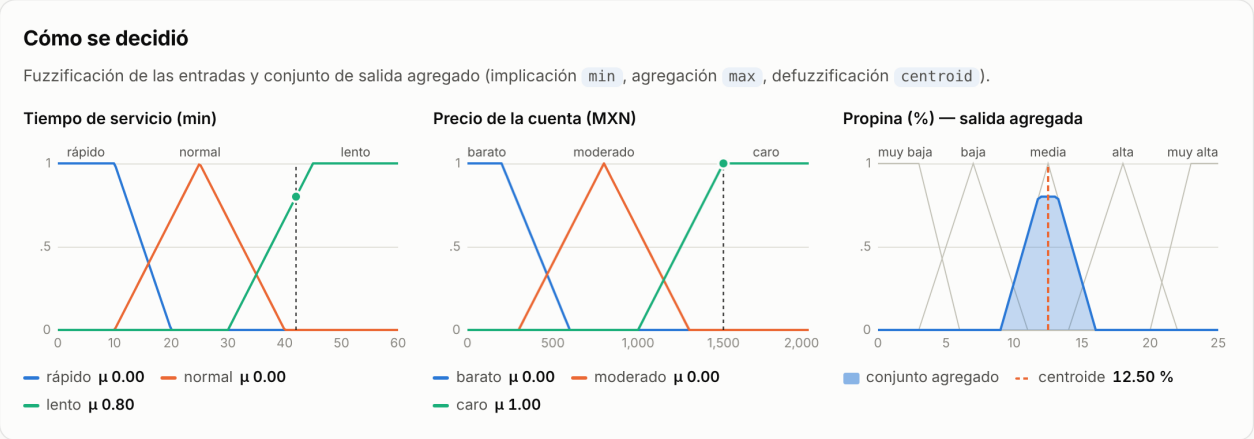
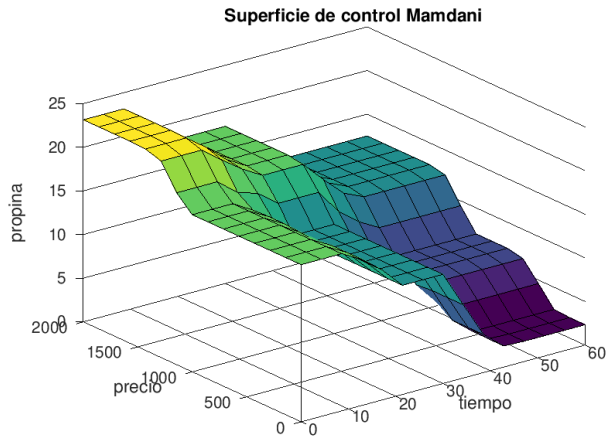
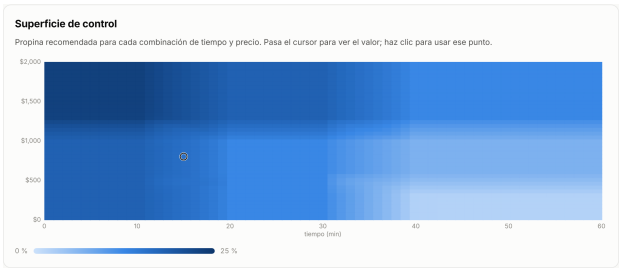


Figura 9: Caso 2 en la calculadora: sólo dispara la regla R9.

8 Superficie de control y casos de prueba



(a) `gensurf` en Octave.



(b) Mapa de calor en la calculadora.

Figura 10: Superficie de control: propina recomendada para cada combinación de tiempo y precio.

La superficie confirma el diseño: la propina disminuye conforme aumenta el tiempo y, para un mismo tiempo, aumenta con el precio. Las mesetas corresponden a zonas donde una sola regla domina; las rampas, a las transiciones entre conjuntos.

Tabla 6: Casos de prueba. La columna *Calculadora* usa el centroide discreto de MATLAB (ecuación 5); la columna *Octave* es la ejecución independiente de `propina.fis` con `evalfis`.

tiempo (min)	precio (MXN)	situación	Calculadora (%)	Octave (%)	diferencia
5	150	rápido, barato	18.00	18.00	0
5	800	rápido, moderado	18.00	18.00	0
5	1800	rápido, caro	23.21	23.15	0.06
25	150	normal, barato	12.50	12.50	0
25	800	normal, moderado	12.50	12.50	0
25	1800	normal, caro	18.00	18.00	0
50	150	lento, barato	2.27	2.33	−0.06
50	800	lento, moderado	7.00	7.00	0
50	1800	lento, caro	12.50	12.50	0
15	450	transición	15.60	15.60	0
15	800	caso 1	15.89	15.89	0
35	1100	transición	11.66	11.66	0
42	1500	caso 2	12.50	12.50	0

8.1 Verificación

Para validar la calculadora se evaluó `propina.fis` en una malla de $61 \times 41 = 2501$ puntos (tiempo de 0 a 60 cada minuto, precio de 0 a 2000 cada \$50) con dos implementaciones independientes: el motor web y `evalfis` de Octave.

- Las únicas diferencias aparecen cuando el conjunto agregado toca un extremo del universo de salida (0 % o 25 %), con un máximo de 0.09 puntos porcentuales. Se deben a que Octave calcula el centroide por integración trapezoidal, $\int y\,\mu\,dy / \int \mu\,dy$ con `trapz`, que da la mitad de peso a los puntos extremos, mientras que MATLAB y la calculadora usan la suma discreta de la ecuación (5).
- Si el motor web usa también `trapz`, la diferencia máxima con Octave en los 2501 puntos es de 5×10^{-15}, es decir, error de redondeo. Por lo tanto, la fuzzificación, la fuerza de disparo, la implicación y la agregación coinciden exactamente.

El paquete de Octave exige $a < b$ en `trapmf`, mientras que MATLAB acepta hombros con $a = b$. Para la verificación los hombros se extendieron una unidad fuera del rango (por ejemplo, $[0\ 0\ 10\ 20] \to [-1\ 0\ 10\ 20]$), lo que no cambia ningún valor dentro del universo de discurso.

9 Obtención de datos en MATLAB

El script `matlab/exportar_datos.m` reúne los comandos para extraer todos los datos del sistema Mamdani. Los principales son:

Código 3: Extracción de datos con `evalfis`.

```
fis = readfis('propina.fis');
opts = evalfisOptions('NumSamplePoints',101);

% Salida para un caso
p = evalfis(fis,[15 800],opts)  % 15.8859

% Detalle de la inferencia (ruleFiring requiere R2019b o posterior)
[p,fuzzIn,ruleOut,aggOut,ruleFiring] = evalfis(fis,[15 800],opts);
```

```
% fuzzIn : grados de pertenencia por regla y entrada (9x2)
% ruleFiring : fuerza de disparo de cada regla (9x1)
% aggOut : conjunto agregado en los 101 puntos

% Superficie completa a CSV
[T,P] = meshgrid(0:1:60, 0:50:2000);
Z = reshape(evalfis(fis,[T(:) P(:)],opts), size(T));
writetable(table(T(:),P(:),Z(:),'VariableNames', ...
    {'tiempo_min','precio_mxn','propina_pct'}),'superficie_propina.csv');

% Graficas
plotmf(fis,'input',1); plotmf(fis,'input',2); plotmf(fis,'output',1);
gensurf(fis); plotfis(fis); ruleview(fis)
```

10 Conclusiones

1. El sistema Mamdani traduce reglas expresadas en lenguaje natural en una recomendación numérica continua: con 3 conjuntos por entrada, 5 en la salida y 9 reglas se obtiene una superficie de control suave entre 2.27 % y 23.21 %.

2. La base de reglas captura la relación precio–tiempo: para un servicio lento la propina pasa de 2.27 % con una cuenta barata a 12.50 % con una cara, porque una cuenta cara tolera más espera.

3. Las salidas intermedias de `evalfis` (grados de pertenencia, fuerza de disparo y conjunto agregado) permiten reconstruir a mano cada recomendación, como en el caso de 15 min y \$800, cuyo centroide es 15.89 %.

4. La calculadora web lee el mismo archivo `.fis` que MATLAB y reproduce su inferencia; al compararla con una ejecución independiente en Octave sobre 2501 puntos sólo difiere por el método numérico del centroide en los extremos del universo de salida.

Referencias

[1] L. A. Zadeh, "Fuzzy sets," *Information and Control*, vol. 8, no. 3, pp. 338–353, 1965.

[2] E. H. Mamdani y S. Assilian, "An experiment in linguistic synthesis with a fuzzy logic controller," *International Journal of Man-Machine Studies*, vol. 7, no. 1, pp. 1–13, 1975.

[3] T. J. Ross, *Fuzzy Logic with Engineering Applications*, 4.ª ed. Chichester: Wiley, 2016.

[4] The MathWorks, Inc., *Fuzzy Logic Toolbox User's Guide*. Natick, MA. Disponible en `https://www.mathworks.com/help/fuzzy/`.

[5] L. Markowsky, *Fuzzy Logic Toolkit for GNU Octave*, versión 0.4.6. Disponible en `https://gnu-octave.github.io/packages/fuzzy-logic-toolkit/`.

[6] Anthropic, *Claude* (asistente de inteligencia artificial, usado mediante Claude Code), 2026. Utilizado como apoyo en el diseño del sistema difuso, la programación de la calculadora web, la verificación en Octave y la redacción de este reporte. Disponible en `https://claude.ai`.